CSc 103 Midterm Exam

Instructions

Read each question carefully. Calculators, books, phones, computers, notes, etc. are not permitted. All you need is something to write with. Scratch paper is provided at the front of the room. You'll have 60 minutes for the exam. **NOTE:** Don't use library functions that trivialize problems. See page 6 for a list of functions that are OK to use (or if you need a reminder on something), and be sure to ask before using anything that isn't on the list.

Name: _____

Problem	Score
1 (10 points)	
2 (10 points)	
3 (10 points)	
4 (extra credit) (5 points)	
Total: 30 points	

1. (10 points) Write a small program (a `main()` function) that reads integers from standard input, and prints the smallest integer that is divisible by 3. For example, if the input is `5 9 2 6 7`, then the answer would be 6, since it is smallest among `9 6`, the only inputs that were divisible by 3. If no inputs are divisible by 3, your program should output `INT_MAX`. (See page 6 if you need a reminder about that constant.)

   You can assume that all the standard `#includes` are there. Just write the `main()` function. (**NOTE:** You don't even need vectors for this, and in fact should NOT use them.)

```
int main()
{
    int s = INT_MAX;
    int x; // input
    while(cin >> x) {
        if(x < s && x % 3 == 0)  s = x;
    }

    cout << s << "\n";
    return 0;
}
```

2. (10 points) Write a function that evaluates a polynomial f at a given input x. **NOTE:** we will describe the polynomial f by a list of its roots, which assume are all real. That is, the polynomial $f(x)$ has the following form, where r_i are the roots of f:

$$f(x) = (x - r_0)(x - r_1)\dots(x - r_{n-1}).$$

For example, if the input roots are $2, -3, 4$ and $x = 1$, then the output should be $12 = (1-2)(1+3)(1-4)$.

Here is a start, where vector `r` contains the roots:

```
int polyEval(const vector<int>& r, int x) {
    int a = 1; // will be the answer
    for(size_t i = 0; i < r.size(); i++)
        a *= (x - r[i]);
    return a;
}
```

3. (10 points) Write a function that takes 2 vectors of **sorted** integers, and merges them into a third **sorted** vector which you can assume is initially empty. The prototype should look like this (note that the last reference parameter is for the *output*, which is why it is not `const`):

```
void merge(const vector<int>& V1, const vector<int>& V2, vector<int>& result);
```

As an example, if the first two vectors contain $[2, 4, 5, 9, 13, 17]$ and $[1, 3, 7, 8]$ respectively, then the result vector should contain $[1, 2, 3, 4, 5, 7, 8, 9, 13, 17]$. **Note: Do NOT use a sort function (even if you write it yourself). You'll not receive any credit for doing the problem this way.**

2	4	5	9	13	17

i

1	3	7	8

j

i+7

i=10

i++ vs ++i

10 11

result = | 1 | 2 | 3 | 4 | 5 | 7 | 8 |

Details:

```
size_t i=0, j=0;
size_t n=V1.size(), m=V2.size();
while (i<n && j<m) {
    if (V1[i] < V2[j]) result.push_back(V1[i++]);
    else               result.push_back(V2[j++]);
}
while(i<n) result.push_back(V1[i++]);
while(j<m) result.push_back(V2[j++]);
```

4. (5 points) Write a function that takes a vector of integers V and a target integer t, and returns a boolean value indicating whether or not there are two distinct elements of V such that $V[i] + V[j] = t$. **Important:** your algorithm should NOT perform a brute force search. If n is the size of V, it must take fewer than cn^2 steps asymptotically, for any value of c.

   Hint: begin by taking for granted a sorting algorithm that works in fewer than cn^2 steps. If you solve the problem using this mystery sorting algorithm, that will get you 4/5 points. Here's a prototype for such a sort function:

```
void sort(vector<int>& V);
```

Cheat Sheet / Function Reference

Linked List Structure

```
struct node {
  int data;
  node* next;
  node(int d=0, node* n=NULL) : data(d),next(n) {}
};
```

Vectors

Use any of the following `vector` functions:

- `push_back(x)` (adds `x` to the end of the vector)
- `pop_back()` (removes whatever is at the end of the vector)
- `size()` (returns the number of elements)
- `resize(n)` and `resize(n,x)` (forces vector to have size `n`; if a second parameter `x` is given, any new values will be set to copies of `x`)
- `clear()` (remove all elements)
- `back()` (this gives the last element; `V.back()` is the same as `V[V.size()-1]`)

Strings

The `string` type actually supports **all** of the above vector functions, but you can also use the following:

- `x + y` (gives a new string that is the concatenation of strings `x` and `y`)
- `x += y` (appends string `y` to the end of `x`)
- `length()` (returns the number of characters; synonym for `size()`)

Constants

If you need the smallest thing that can be stored in an integer, it is `INT_MIN`; the largest is `INT_MAX`.

General

Also remember that you can access the elements of vectors and strings using the square brackets, e.g., `V[i]`, and that making an assignment between vectors or strings does what you expect (left hand side becomes a copy of the right hand side). You can also check vectors and strings for equality with `==` or other comparison operators like `<,>` (doesn't always make

sense for vectors, though). And if you know about constructors, you can use those as well, but they will never be essential (might just shorten your solution a bit). Also, if you need to read all integers from `stdin`, remember you can just do this:

```
int x;
while (cin >> x) {
        /* do something with x... */
}
```

Or to read all lines of text into strings:

```
string s;
while (getline(cin,s)) {
        /* do something with s... */
}
```